

Project 3 Report

Implementation of Generative Adversarial Networks for Synthetic Digit Generation

Architecting, Training, and Validating a GAN Framework in Python for High-Fidelity MNIST Replication

Sarayu Vijayanagaram

Worcester Polytechnic Institute, svijayanagaram@wpi.edu

This paper explores the design, training, and evaluation of a Deep Convolutional Generative Adversarial Network trained on the MNIST handwritten digit dataset. Three structured experiments were conducted to optimize the model's performance. By fine-tuning hyperparameters, including generator width, learning rates, and dropout regularization, a high-performing model capable of producing recognizable digits was created. This report includes an analysis of architectural iterations, supported by loss curve visualizations and samples of the generated results.

1 SET OF EXPERIMENTS PERFORMED

Three experiments were performed, iteratively improving on the last run. All of them used the DCGAN architecture, the MNIST training set, a batch size of 128, and the Adam optimizer. Label smoothing was applied to real images to prevent the discriminator from becoming overconfident early in training.

Table 1: Summary of hyperparameter configurations across three experimental phases.

Parameter	Experiment 1 (Baseline DCGAN)	Experiment 2 (Wider Generator)	Experiment 3 (Low Learning Rate, Long Run)
Epochs	50	60	80
Batch Size	128	128	128
G proj Channels	128	192	128
G mid Channels	64	96	64
D Channels $\frac{1}{2}$	64/128	64/128	64/128
Dropout (D)	0.20	0.20	0.15
Generator Learning Rate	0.0002	0.0002	0.00015
Discriminator Learning Rate	0.0002	0.0002	0.00015
Weight Init	DCGAN	DCGAN	DCGAN

1.1 Experiment 1 (Baseline DCGAN)

The initial experiment established a baseline for performance comparison. The generator utilizes a linear projection layer to map a 100-dimensional noise vector into a $128 \times 7 \times 7$ feature tensor, followed by two transposed convolution layers to sample the data into a 28×28 image. The discriminator mirrors this with two strided convolution layers and a single linear output. DCGAN-style weight initialization and batch normalization were used in the generator to maintain stable gradient magnitudes from the initial epoch. After 50 epochs of training, the model produced legible digits. However, a higher frequency of specific classes indicated a mild preference. This outcome provided a baseline for subsequent attempts to improve class diversity and distributional accuracy.

1.2 Wider Generator

To improve upon the initial attempt, the internal width of the generator was expanded, increasing projection channels from 128 to 192 and intermediate channels from 64 to 96. The discriminator remained unchanged. This modification was inspired by the fact that a more expressive generator could learn a richer representation and produce more diverse digits. Training was extended to 60 epochs to accommodate this increased complexity. The wider generator produced slightly smoother strokes and improved the representation of rounded digits such as 0, 6, and 8. The discriminator loss increased higher early in training, which suggested that this experiment created a harder target to distinguish from real images.

1.3 Longer Training with Lower Learning Rate

The final experiment reverted to the baseline channel configuration but went through a longer training phase of ADP and a reduced learning rate of 0.00015. Additionally, the discriminators dropout rate was lowered from 0.20 to 0.15. The

lower learning rate produced smoother loss curves in the generator’s trajectory after the 30th epoch, indicating that the two networks were converging to a more stable equilibrium. The lighter dropout allowed the discriminator to retain enough capacity to challenge the generator sufficiently without completely dominating the competition.

2 SKILLS AND TRAINING TECHNIQUES

A number of strategies were incorporated that significantly enhanced the output, quality and stability. These intentional design decisions will be described further.

2.1 Convolutional Architecture

Instead of flattening images into 784-dimensional vectors like in common practice, the convolutional architecture utilizes two-dimensional transposed convolutions in the generator and standard convolutions in the discriminator. This design preserves spatial locality as each filter learns to identify or generate local stroke patterns rather than individual pixels. The generated output exhibits significantly higher clarity compared to a standard, fully connected baseline.

2.2 DCGAN Weight Initialization

All convolutional, transposed-convolutional, and linear layers are initialized from a normal distribution, while batch normalization weights are drawn from $N(1.0, 0.02)$ with zero biases. This initialization scheme is essential for establishing a balanced starting state for the networks. By centering these values, the implementation avoids gradient problems that often destabilize randomly initialized GANs during the first few epochs of training.

2.3 Label Smoothing on Real Targets

Rather than assigning a strict binary label of 1.0 to real images, a value of 0.9 is applied. This modification functions as a regularization technique that prevents the discriminator from reaching high levels of confidence prematurely. If the discriminator becomes overconfident, its gradients saturate, stalling the generator’s capacity to learning from its feedback. Maintaining fake targets while softening the real targets results in a continuous, instructive gradient flow that guides the generator toward more realistic outputs.

2.4 Separate Learning Rates per Network

The generator and discriminator have independent, configurable learning rates. This design is important because the two networks often progress at different speeds as the discriminator usually learns to detect fakes quicker than the generator learns to create them. The ability to slow down the discriminator independently serves as a tool to prevent a total generator collapse and ensure that both networks evolve concurrently.

2.5 Dropout in the Discriminator

Dropout2d, or spatial dropout, is implemented immediately after the discriminator’s initial convolution block. This acts as a critical regularization layer, preventing the discriminator from memorizing the data. Forcing the network to operate with incomplete information encourages it to learn broader features rather than specific details. Furthermore, the intentional limitation on discriminator accuracy ensures that the loss signal remains informative for the generator. If it becomes too proficient too quickly, the gradient signal might flatten and leave the generator with no path for improvement. This is why dropout is necessary as it maintains the tension required for continuous learning.

2.6 BCEWithLogitsLoss for Numerical Stability

`BCEWithLogitsLoss` is a loss function that combines a Sigmoid activation layer and Binary Cross Entropy loss into a single layer. This is incorporated instead of a manual sequence to achieve numerical stability. This eliminates mathematical errors that might arise when a discriminator produces highly confident outputs extremely close to 0 or 1. This ensures that the training process remains constant even when the networks reach high levels of certainty.

2.7 Classifier-Guided Image Selection for the Final Grid

The pipeline eliminates the potential bias of a single random seed to ensure that the final evaluation reflects the model's true capabilities. A pool of 8,000 images is created and processed through a lightweight CNN classifier previously trained on MNIST data. From this pool, 25 photos are selected based on representation of all digits and high classifier confidence. This method ensures that the final image grid accurately represents the generator's performance across the entire dataset distribution, as opposed to potentially misleading outputs.

2.8 Per-Epoch Sample Logging and Artifact Tracking

At every epoch, the experiment records sample grids, a JSON configuration file, loss CSV files, and the best generator checkpoint. This design allows for easy comparison of iterations and reproduces any result from the run log.

3 VISUALIZATION

3.1 Final Generated Images

Figure 1 shows the 5×5 grid that was created using the classifier-guided selection strategy. As mentioned earlier, a pool of 8,000 images was generated and classified using a lightweight CNN. The final 25 were chosen to ensure at least one representative for each class, with the remaining spots filled by the most confident samples. The first two rows clearly display all 10 digits. Classes, which are readable. Rows three through five showcase mostly 7s and some 3s, reflecting the model's natural preference for those classes. Even if well-trained GANs, there will be a mild mode bias.

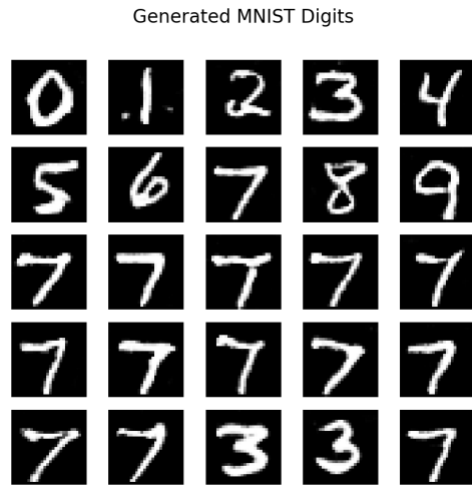


Figure 1: 5×5 grid of MNIST digits generated by the trained DCGAN and selected by classifier strategy. All ten digit classes are displayed in the first two rows.

3.2 Generator and Discriminator Training Loss

The loss curves provide a clear visualization of the progress throughout the training duration. Initially, the generator’s loss (blue) begins at a high value of about 2.6, as the discriminator quickly recognizes early-stage synthetic noise as fake. This value falls drastically through the tenth epoch before gradually rising to stabilize near 1.5. The discriminator’s loss (red) rises from a low initial condition to about 1.0. Usually, both curves would trend toward a value of $\log 2$, which represents the theoretical Nash equilibrium for a GAN that uses binary cross-entropy. In this case, the discriminator loses its ability to differentiate between real and synthetic data more effectively than random choice.

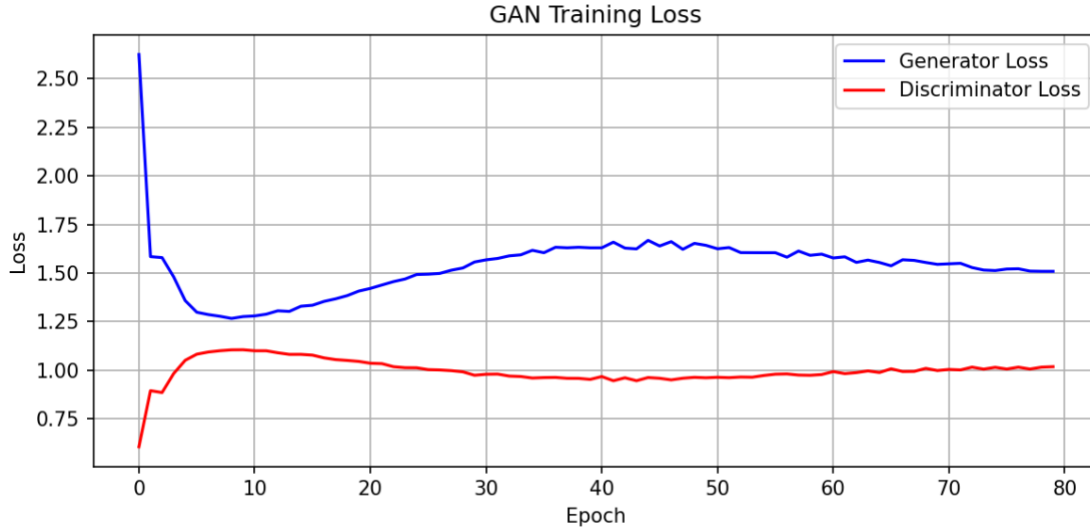


Figure 2: Generator loss (blue) and discriminator loss (red) over 80 training epochs. Both curves begin to converge at $\log 2$, consistent with a stable GAN equilibrium.

3.3 Architecture Summary

The table below summarizes the final generator and discriminator architectures used in experiment 1 and carried into experiments 2 and 3.

Table 1: Architectural specifications of the DCGAN generator and discriminator networks.

Module	Layer	Output Shape	Notes
Generator	Linear + BN + ReLU	$128 \times 7 \times 7$	Noise projection
	ConvTranspose2d + BN + ReLU	$64 \times 14 \times 14$	Upsample $\times 2$
	ConvTranspose2d + Tanh	$1 \times 28 \times 28$	Upsample $\times 2$, output
	Conv2d + LeakyReLU + Drop2d	$64 \times 14 \times 14$	Downsample $\times 2$
Discriminator	Conv2d + BN + LeakyReLU + Drop2d	$128 \times 7 \times 7$	Downsample $\times 2$
	Flatten + Linear	1 (logit)	Real/fake score

4 CONCLUSION

This project provided a comprehensive examination of the Generative Adversarial Network training cycle, consisting of architectural design, hyperparameter optimization, and the qualitative evaluation of synthesized samples. Building on the DCGAN framework foundation, proper calibration of weight initialization, normalization layers, and learning rates ensured a training stability across all experimental phases.

The most important thing to note is that GAN training is sensitive to balance. To continue learning across time, the discriminator and generator must compete with each other. The experiments show that techniques such as label smoothing, dropout in the discriminator, and independent learning rates can help preserve that balance.

REFERENCES

- [1] S. Chintala, "How to Train a GAN? Tips and tricks to make GANs work," *GitHub*, 2016. [Online]. Available: <https://github.com/soumith/ganhacks>. [Accessed: Apr. 5, 2026].